

Macrodork hardware diagrams

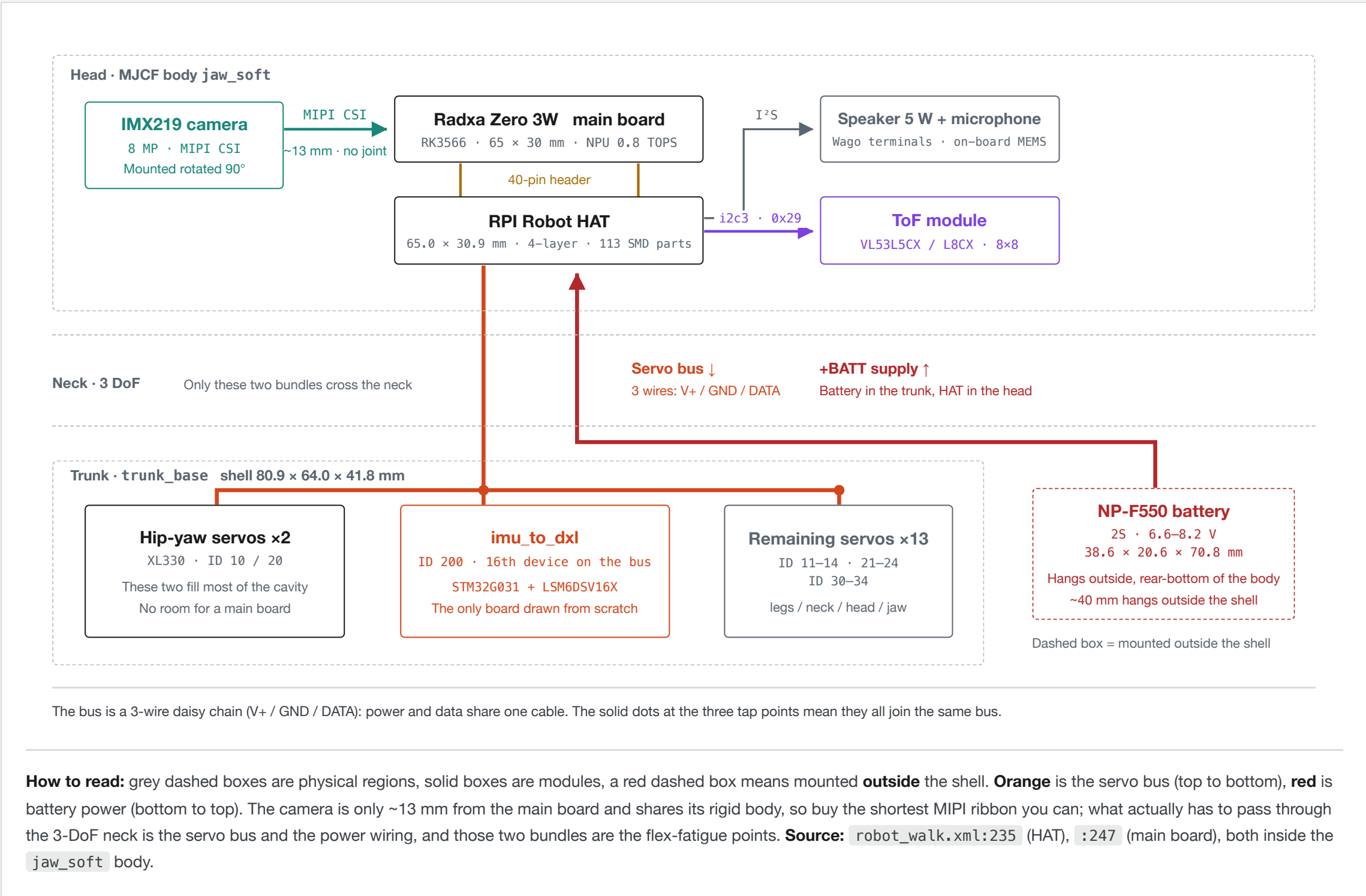
Five modules, two home-made boards, one bus. Every pin, address and register is taken from the official open-source hardware project and the runtime source code; none of it has been verified on real hardware.

— Servo bus · half-duplex TTL 1 Mbps — +BATT 6.6–8.2 V — +5V / +3V3 — I²C · 400 kHz — MIPI CSI — I²S audio

Bus devices 16 15 servos + 1 IMU board	Control cycle 20 ms 50 Hz, one read, one write	Home-made boards 2 HAT is open source / IMU board must be drawn	Policy network 775 KiB ×9 ONNX files
---	---	--	---

01 Where things live

The easiest thing to get wrong: **the main board is not in the trunk, it is in the head.** Camera, main board and HAT all belong to the same MJCF body, `jaw_soft` ; the camera sits about 13 mm from the centre of the main board with no joint in between. The trunk cavity is almost entirely filled by the two hip-yaw servos, and the battery hangs outside, below the rear of the body.

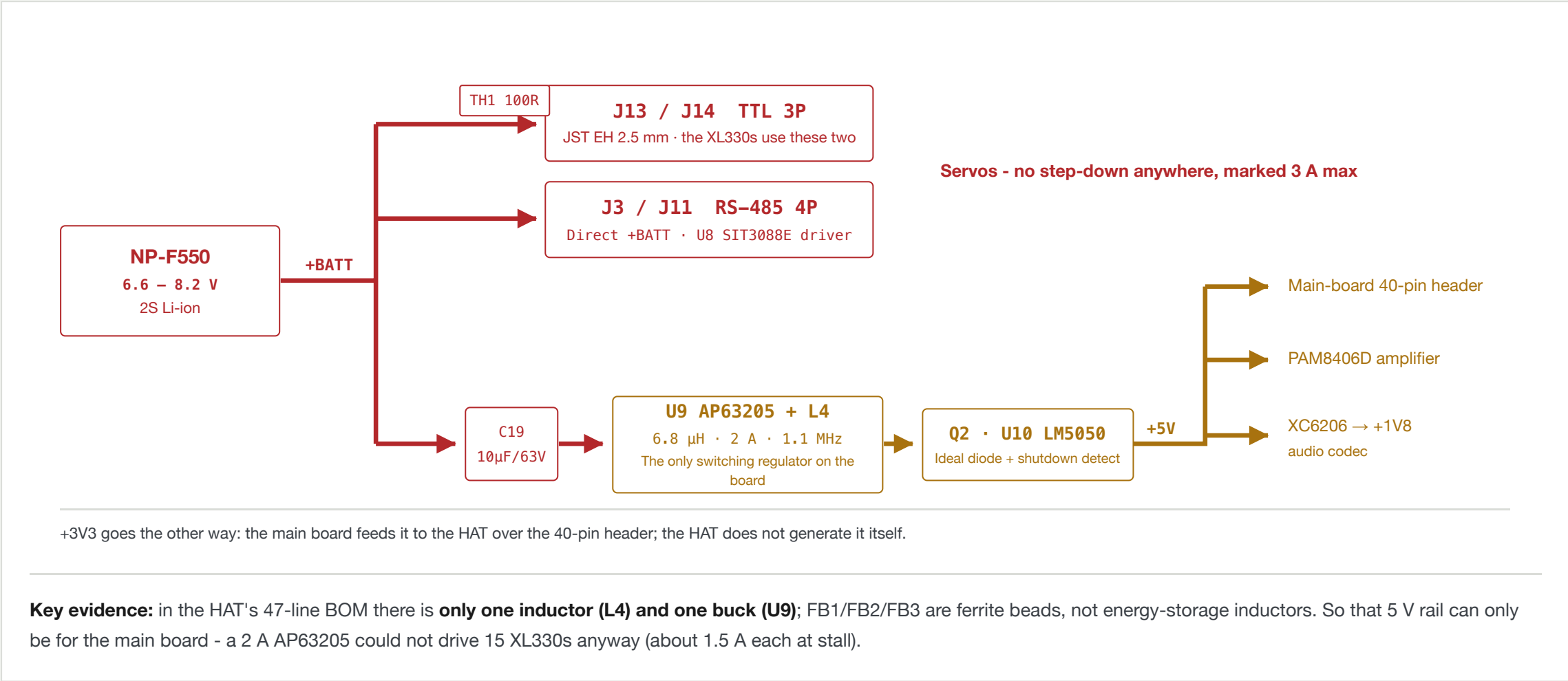


READ THIS BEFORE BUYING

Because the main board lives in the head and the slot is only 65 × 30 mm, no larger board will fit - the LCSC Taishan Pi is **70 × 45 mm**, the same RK3566 but it does not go in, and a HAT still has to sit on top of it.

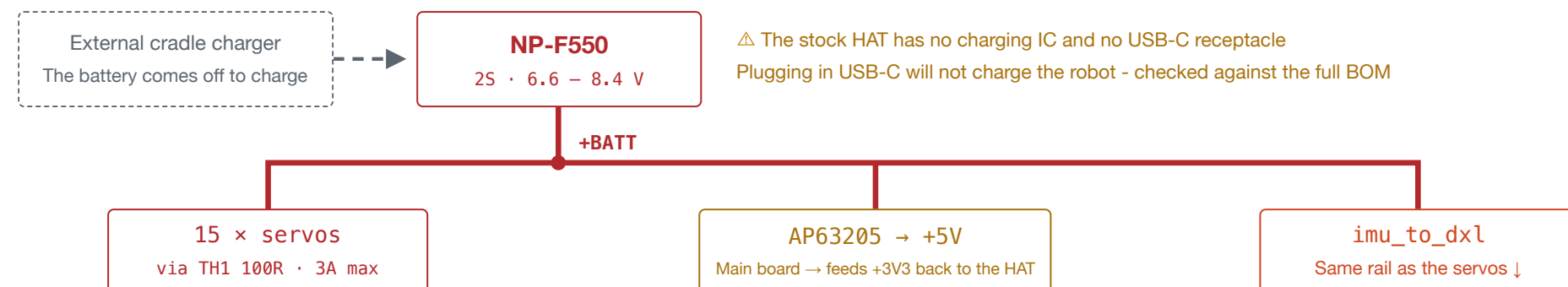
02 The battery feeds two rails; the servos run on raw battery voltage

This is the most counter-intuitive thing on the whole HAT: **the servos are wired to raw battery voltage, with no step-down at all.** The XL330 is officially rated 3.7–6.0 V, recommended 5.0 V, yet here it gets 6.6–8.2 V. That is not a typo; it is a deliberate upstream choice - more stall torque, paid for in heat and lifetime.

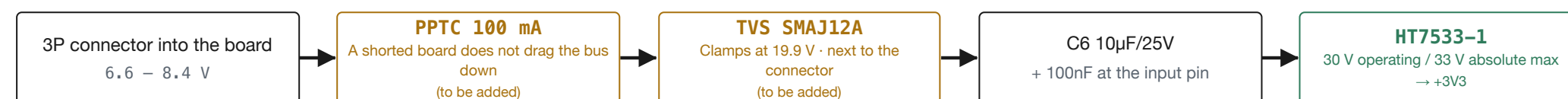


Supply, charging and over-voltage protection

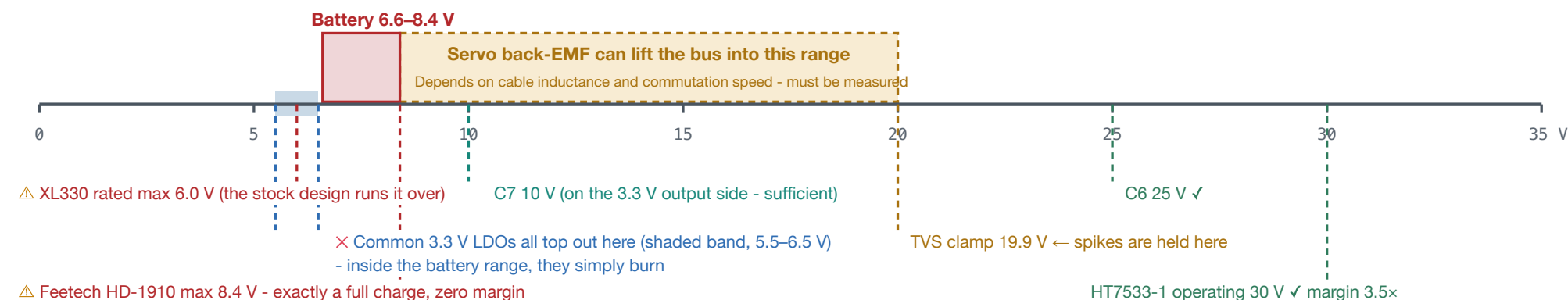
Charging - there is no charging circuit on the board



imu_to_dxl input protection chain - it shares the bus with the servos and will see back-EMF



Voltage window - the bottleneck at a glance



Three lines of defence: a wide-input LDO (chosen) blocks sustained over-voltage · a TVS (to be added) clamps transient spikes · a PPTC (to be added) keeps a fault on this board from taking the whole bus down. A series diode guards against reverse polarity; it does nothing for forward over-voltage.

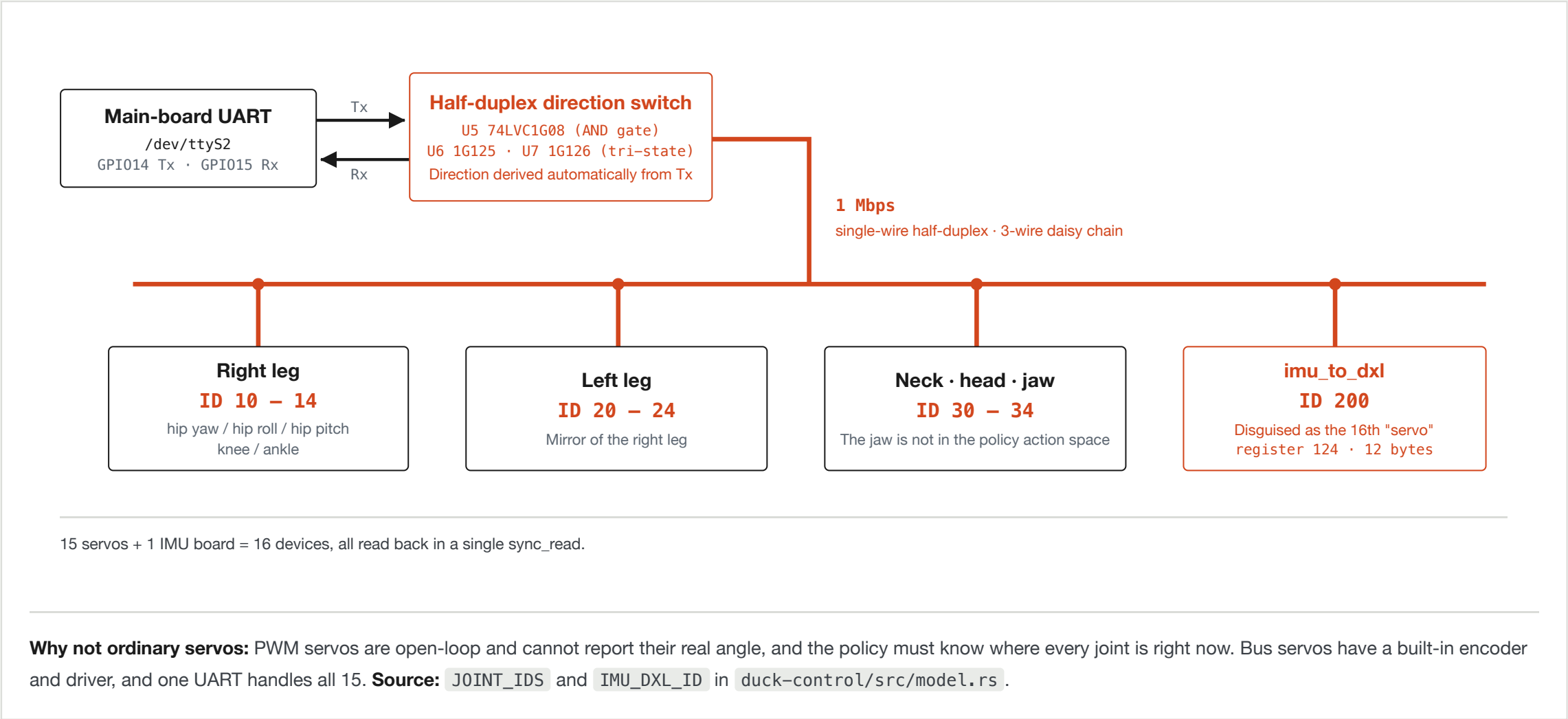
How to read: above the axis are the **voltages that will actually occur**, below it are the **voltage ratings of each part**. Whenever a line below falls inside a range above, that part is the bottleneck. **Common 3.3 V LDOs all land inside the battery range - they simply burn.** △ **The back-EMF magnitude is illustrative, not measured** - it depends on cable inductance and commutation speed; put a scope on it once the parts arrive.

THIS EXPLAINS THE VOLTAGE ADAPTATION IN THE FIRMWARE

Because the servos run straight off the battery, the same command produces very different torque between a full and a flat pack. At runtime the compensation is $\text{scale} \times (7.4 / \text{measured voltage EMA})$; without it the policy's behaviour drifts with the state of charge.

03 Sixteen devices on one bus

This is the central argument of the whole design: **15 servos are daisy-chained on one three-wire bus, and the IMU also masquerades as a device on that bus.** So the main board issues a single `sync_read` per cycle and gets body attitude and all 15 joint angles back together - timestamps aligned by construction, with no software juggling of jitter between two separate data paths.



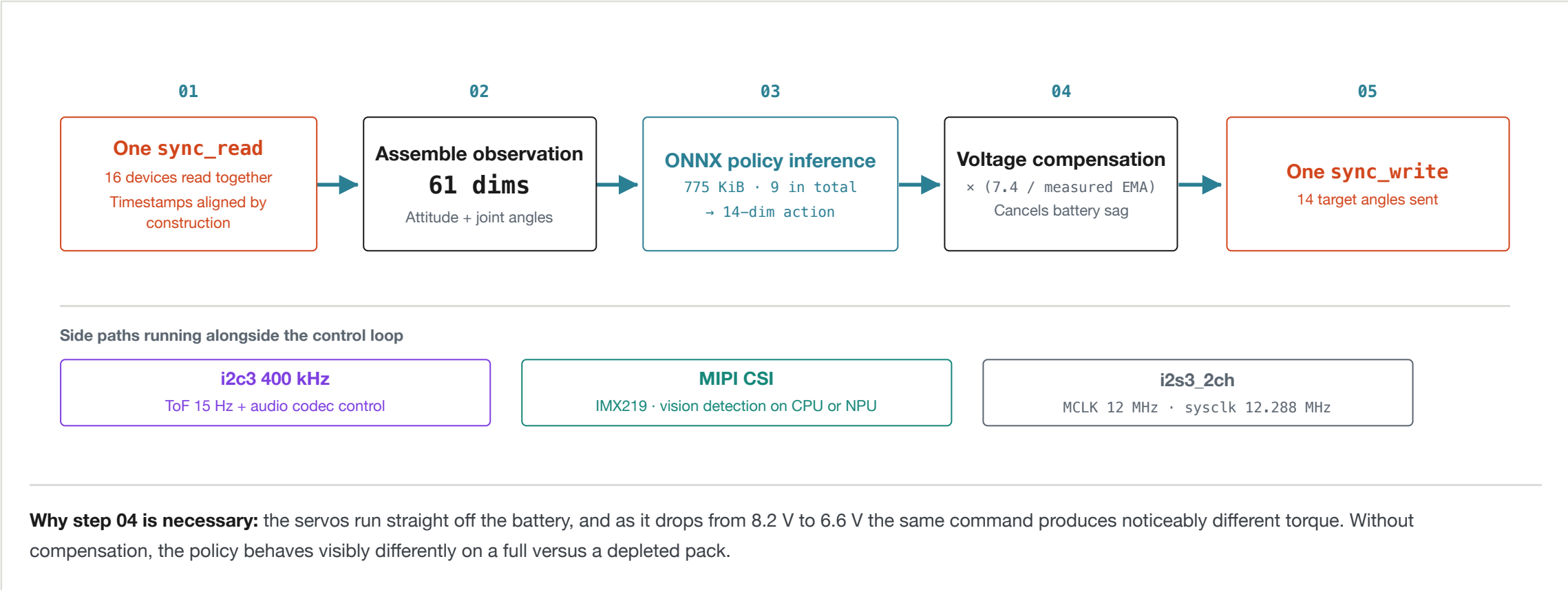
THE 20 MS BUDGET IS TIGHTER THAN IT LOOKS

The XL330 ships with `return_delay_time = 250`, i.e. a **500 μ s bus-turnaround delay per device**. Quoting the source: "Across 16 devices that is **8 ms per tick — 40% of a 20 ms budget**". The upstream fix is to write 0 to that register.

Whichever servo you switch to, this step has to be redone and measured - it is not one vendor's problem.

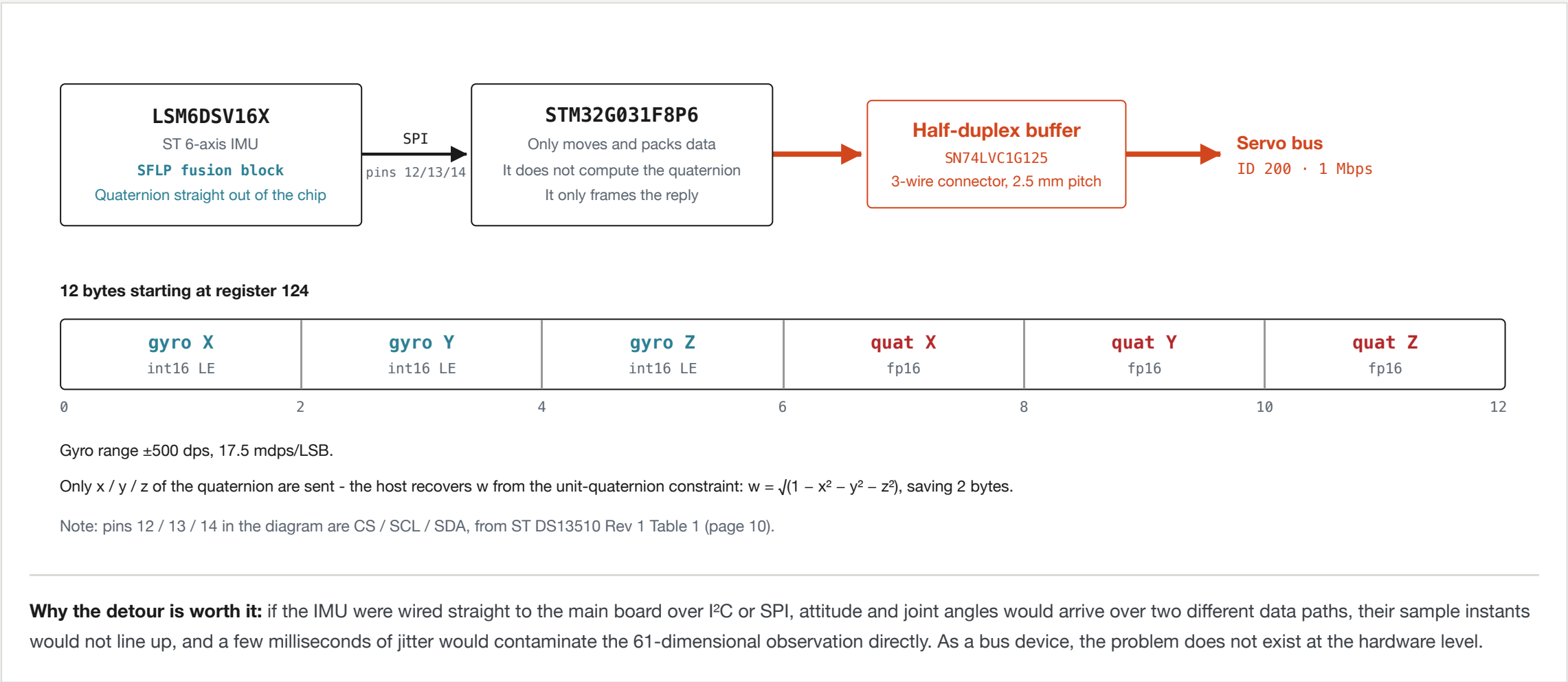
04 One cycle, 20 milliseconds

The control loop has only five steps, and they happen in this order. The numbers here are the real sequence, not typographic decoration.



05 The board you have to draw yourself

Its entire job is three things: read the IMU, pack the data into 12 bytes, and hand those 12 bytes over when the host issues a `sync_read` . Upstream has not open-sourced this board, but **the protocol and register layout can be reconstructed in full from the runtime source** - the code is the spec.



06 40-pin usage and key addresses

PIN / BUS	PURPOSE	LINUX DEVICE	SOURCE
GPI014 / GPI015	Servo bus Tx / Rx	/dev/ttyS2 @ 1 Mbps	robotd.toml
GPI002 / GPI003	I²C - IMU, audio codec, Stemma	i2c3 @ 400 kHz	i2c3-pihat.dts
GPI018 / 19 / 20 / 21	Audio PCM clock / frame sync / in / out	i2s3_2ch	aic3104-i2c3.dts
GPI006	Shutdown detect	—	HAT schematic
GPI022	Interrupt pin of the BMI088 on the HAT - unused by software	—	i2c3-pihat.dts
MIPI CSI	IMX219 camera, mounted rotated 90°	radxa-zero3-rpi-camera-v2	setup-board.sh
i2c3 · 0x18	Audio codec TLV320AIC3104	—	i2c3-pihat.dts
i2c3 · 0x29 / 0x52	ToF, both addresses are tried	—	tof/src/main.rs
i2c3 · 0x19 / 0x68	BMI088 - populated but marked dormant in software	—	i2c3-pihat.dts
Bus ID 200	imu_to_dxl, register 124, 12 bytes long	—	duck-control/model.rs

THIS TABLE IS THE CHECKLIST

The upstream Microduck software is open source, so every address on the hardware has a counterpart in the code. **If they don't match, one side is wrong** - that is the cheapest verification step when replicating.

07 Two routes: stock XL330 versus Feetech STS

The "IMU as the 16th device on the bus" architecture **holds on Feetech too** - because Feetech STS/SCS has its own bulk-read instruction, `SYNC_READ ( 0x82 )`, used exactly the same way as on Dynamixel.

And the reply mechanism is the same on both sides. Protocol V2 does have a `fast_sync_read ( 0x8A )` that merges every reply into one packet, but **the official runtime does not use it** - a repo-wide search for `fast_sync_read / 0x8A` returns nothing, and `duck-control/src/bus.rs:236` calls the plain `sync_read_raw_data`. This is also consistent with the "500 μ s **per device** $\times$ 16 devices = 8 ms" in the source - **if the replies were merged into one packet, the per-device delay would not be multiplied by 16.**



Protocol differences at a glance

ITEM	DYNAMIXEL V2 · XL330	FEETECH STS/SCS
Physical layer	Identical - half-duplex single-wire TTL, 3 wires (V+ / GND / DATA), 1 Mbps	
Connector pitch	JST EH 2.5 mm	5264, also 2.5 mm
Header	FF FF FD 00	FF FF
Checksum	CRC-16	~sum (inverted sum)
Bulk read	sync read (what upstream actually uses) also fast sync read 0x8A, unused	sync read 0x82
Reply scheme	Same - one independent status packet per device	
Bus turnarounds / cycle		17 
 The turnaround count is inferred from the protocol (1 instruction + 16 replies), not measured		
Factory return delay	250 = 500 μs / device, must be written to 0	Default 0 , and not functional on STS
ID range	0 – 252	0 – 253 (register 5)
Baud-rate setting	register <code>baud_rate = 3</code>	register 6, <code>0 = 1M</code>

ONE MORE REGISTER WORTH TUNING ON THE FEETECH SIDE

Register `8 (0x08)` of the STS memory map, **Response Level**, default **1** - "reply to every command". Set it to **0** and only read and ping return a packet; ordinary write instructions no longer take up bus time.

This is the same kind of move as upstream writing 0 to `return_delay_time` on the XL330: **the factory defaults are for general use; a high-rate control loop has to tune them itself.**

What switching to Feetech does to each module

MODULE	IMPACT	NOTES
Main board	Unchanged	Independent of the servo protocol
Camera / ToF	Unchanged	Each uses its own separate interface
HAT	No change at all	Feetech and XL330 have identical physical layers - 3-wire half-duplex TTL, 1 Mbps, 2.5 mm pitch connector. J13 / J14 on the HAT are TTL 3P ports; Feetech servos plug straight in , not a single wire needs to move
imu_to_dx1	Hardware architecture unchanged	Same physical layer, so one hardware revision is enough; the firmware detects the protocol from whether the third header byte is <code>FD</code> , with two parsing branches. Standardise the connector on 2.5 mm pitch and it works with both
Policy	Almost certainly needs retraining	Changing servos changes both the torque curve and the mass; the nine upstream ONNX files cannot be reused as they are. This is the single biggest piece of work on the whole route, and it has nothing to do with the protocol

ANOTHER QUESTION: DOES THIS HAT NEED TO BE BUILT AT ALL?

This is independent of which servo you pick; it is purely a cost call. The HAT is a 4-layer board with SMD assembly - prototyping starts at a few hundred yuan with a minimum order quantity - and **roughly half the circuitry on it is audio** (codec, amplifier, on-board microphone, speaker terminals).

If you are not doing voice features, that half of the board is wasted - in which case a bus adapter board costing twenty-odd yuan plus a buck module covers the two functions that are actually essential, "servo bus" and "power distribution", with the ToF hanging directly off i2c3. **The price is losing on-board audio and Pi HAT+ identification.**

DO THIS BEFORE DRAWING THE BOARD

The turnaround counts above are **protocol facts**, but converting them to milliseconds is an **estimate**. The real number depends on the tri-state buffer switching speed, the inter-packet gaps and the actual register length being read - only a measurement on real hardware will tell.

First thing once the servos arrive: connect a debug board, run both read methods, and time one full round of 16 devices. **Until that number exists, the architectural decisions on the board are spinning in the void.**